Executive Summary

I built an AKS cluster and a private container registry with Terraform, then ran an nginx app on the cluster using an image from that registry. Terraform created a resource group, an Azure Container Registry, and an AKS cluster with Entra ID sign in and Azure RBAC. It also gave the cluster's kubelet identity the AcrPull role on the registry, and nothing else. The nginx pod started with the image acruvuit3200anguun.azurecr.io/nginx:stable, so the cluster pulled it from my private registry. I opened the app with kubectl port-forward.

Architecture Overview

My laptop runs Terraform and kubectl, and everything else lives in one Azure resource group in West US 2.

- **Resource group** rg-uvu-it3200-anguun holds everything.
- **Container registry** acruvuit3200anguun stores the image privately. The admin user is turned off.
- **AKS cluster** aks-uvu-it3200-anguun has one node (Standard_D2s_v6). It uses Entra ID sign in and Azure RBAC for Kubernetes permissions.
- **Kubelet managed identity** is what the node uses to pull images. It has AcrPull on the registry.
- **My user account** has the Cluster Admin role on the cluster so I can deploy.
- **Namespace** demo-app runs the nginx pod and a ClusterIP service.

Terraform Infrastructure

file	What it do
versions.tf	AzureRM and AzureAD providers
variables.tf	Subscription ID, student name, region, VM size, node count, namespace, service principal switch
main.tf service_principal.tf	Resource group, registry, AKS cluster, role Entra app, service principal, secret and its roles (off by default)
outputs.tf	Names, registry login server, kubelet identity ID, tenant ID

k8s/	Namespace, deployment and service manifests

The default VM size is Standard_D2s_v6 because the smaller sizes I tried did not work in this region. Secrets and state are kept out of Git with .gitignore, which excludes terraform.tfvars, state files and the .terraform folder.

Identity and Access Management

This project has four different identities, and each one does a different job.

My Entra user account A person's sign in

Service principal An app identity in Entra ID

AKS cluster managed identity The identity the cluster uses to manage its own Azure resources

AKS kubelet managed identityThe identity the nodes use

Azure uses different identities because the jobs are different. A person signs in interactively, an app signs in with a secret or certificate, and the cluster and its nodes get managed identities that Azure handles for them, so there are no passwords to store. Giving every identity the same permissions would be bad security. If one gets stolen or misused, the attacker gets everything. A node that only needs to pull images should not be able to delete the cluster.

For the service principal, I planned a temporary client secret that lasts 30 days. It would be stored in Terraform state, so the state file is never committed. The secret would be passed in through environment variables and never put in the repo, the report or a screenshot. A better long term option is workload identity, which uses no secret at all. The tenant ID is an output, and the service principal would also give an application ID and an object ID as outputs.

AKS and ACR Integration

The cluster's nodes pull images using the kubelet managed identity, and I gave that identity the AcrPull role on the registry through Terraform. The nodes need this because the registry is private. Without permission, Kubernetes can schedule the pod but the node cannot download the image, so the pod never starts. In main.tf the role assignment uses the cluster's kubelet_identity object ID as the principal and the

registry as the scope. That way it covers this one registry and nothing else. I checked it in the portal: the registry's Access control page shows AcrPull assigned to the aks-uvu-it320... managed identity.

Kubernetes Application Deployment

I deployed nginx into its own namespace using an image stored in my private registry, and the pod ran.

1. Imported the image into the registry: az acr import -n acruvuit3200anguun --source docker.io/library/nginx:stable --image nginx:stable. The portal shows the nginx repository with the stable tag.
2. Got credentials with az aks get-credentials and kubelogin convert-kubeconfig -l azurecli.
3. Created the namespace with kubectl apply -f k8s/namespace.yaml.
4. Applied the deployment with the registry placeholder replaced by acruvuit3200anguun.azurecr.io.
5. Applied the service with kubectl apply -f k8s/service.yaml (ClusterIP, port 80).

Results:

- Node status was Ready.
- The pod web-6d8df88578-bk4gd was Running with 1/1 ready and 0 restarts.
- kubectl get pods -n demo-app -o jsonpath=... printed acruvuit3200anguun.azurecr.io/nginx:stable, which shows the image came from my private registry.
- kubectl get all -n demo-app showed the pod, service, deployment and replica set all healthy.
- After kubectl port-forward -n demo-app svc/web 8080:80, http://localhost:8080 showed the nginx welcome page.

Permission and Security Analysis

Every permission in this project is scoped as small as it can be, except the temporary Cluster Admin role on my own account.

Troubleshooting Findings

Installing kubelogin	winget could not find the package
terraform validate	Missing required argument in the AKS Entra ID block

Conclusion and Lessons Learned

What I learned

Authentication and authorization are two different steps. Signing in successfully does not mean you are allowed to do anything.

The service principal and the kubelet identity are separate. One reaches the Kubernetes API and the other pulls images, and permission on one does nothing for the other.

Subscription Owner is not the same as Entra directory permission. They are different layers.

A VM size being allowed in a region is different from having quota for it.

What I would do differently: check VM quota with az vm list-usage before the first apply, ask about Entra permissions before starting, and deploy without giving myself Cluster Admin so I could test least privilege properly.